

Aonde você quer chegar?

Vai com a



UniCesumar

EAD, SEMIPRESENCIAL E PRESENCIAL



Aula: 09/06/2026
Horário: Das 20h40 às 22h10

Disciplia:

Front End

- **Aula 01:** Introdução ao Desenvolvimento de Front-End
- Aula 02: Criando Páginas com HTLM
- Aula 03: Estilização com CSS e Layout Responsivo
- Aula 04: JavaScript e Interatividade

Introdução ao Desenvolvimento de Front-End

Antes de começarmos...

Quantos **aplicativos** você utilizou hoje?

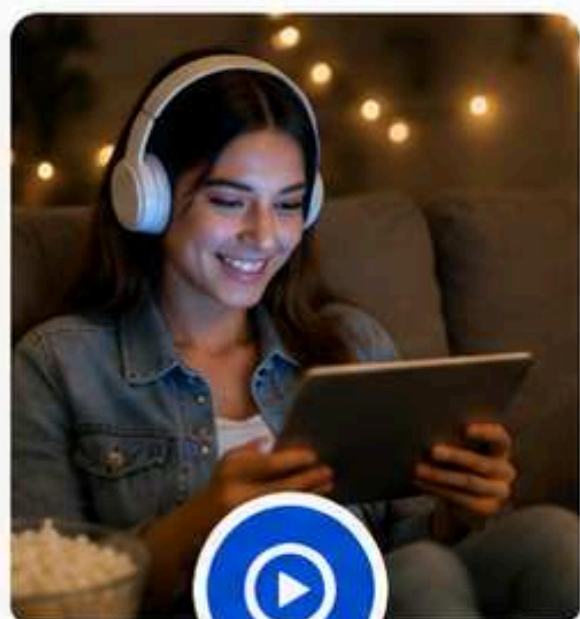
- ☒  WhatsApp
- ☒  Instagram
- ☒  YouTube
- ☒  Banco
- ☒  Uber
- ☒  iFood



O que todos eles possuem
em comum?

A Web está em todo lugar!

A internet conecta pessoas, empresas e serviços, permitindo que **tudo isso aconteça**.



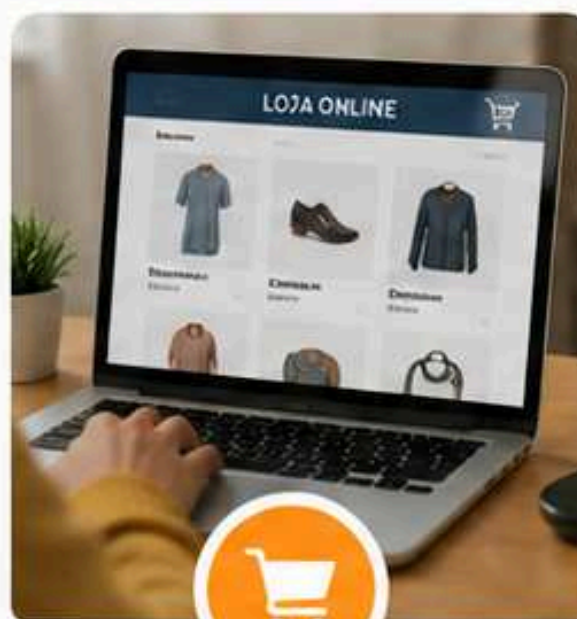
Entretenimento

Filmes, séries, músicas e jogos online.



Serviços Financeiros

Transações bancárias, pagamentos e investimentos.



Compras

Lojas virtuais e marketplaces.



Educação

Cursos online, aulas e aprendizado à distância.



Redes Sociais

Conexão, compartilhamento e comunicação.



Saúde

Consultas, exames e acompanhamento online.



Quase **tudo** hoje passa por uma **interface web**.

O que é Desenvolvimento Web?

Desenvolvimento Web é o processo de criar aplicações e sites que funcionam **através da Internet**.



Essas aplicações podem ser acessadas de **qualquer lugar** e em **diversos dispositivos**.

Exemplos de aplicações web:

NETFLIX

Streaming



E-mail



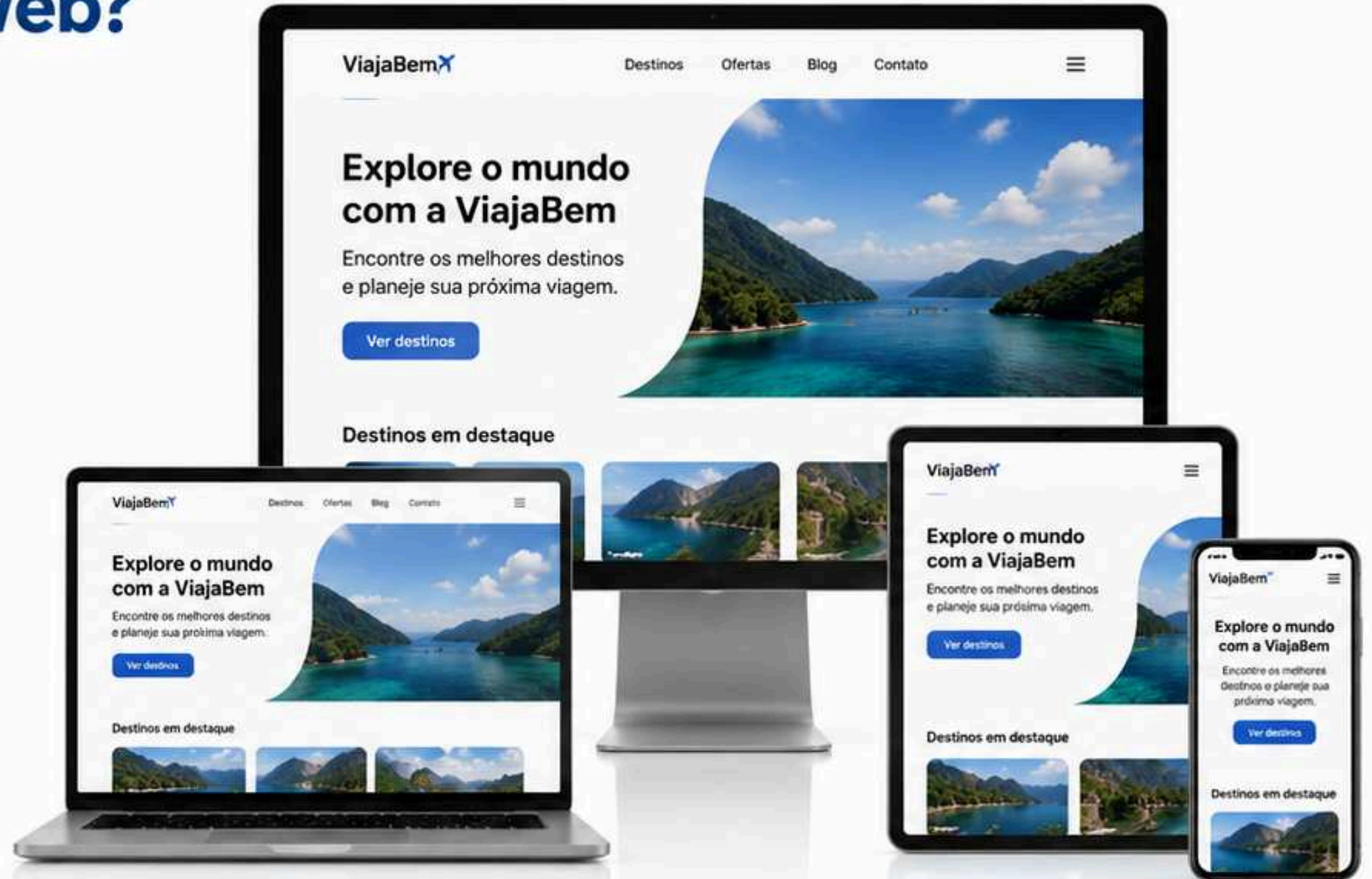
mercado
livre

E-commerce



Spotify

Música



Tudo o que **vemos, clicamos e interagimos** na web é resultado do trabalho de **desenvolvedores web**!

Internet x Web

Entenda a diferença entre a **infraestrutura** que conecta o mundo e o **serviço** que acessamos todos os dias.

 INTERNET	 WEB
 Infraestrutura É a rede mundial que conecta computadores e dispositivos.	 Serviço É um serviço que funciona sobre a Internet.
 Rede Conjunto de equipamentos, cabos, satélites, roteadores e protocolos.	 Conteúdo Conjunto de páginas, sites, aplicativos e recursos acessados por navegadores.
 Função Permite a comunicação e a troca de dados entre dispositivos.	 Função Permite que os usuários visualizem e interajam com informações e serviços online.
 Exemplo Uma “estrada” global que liga tudo ao redor do planeta.	 Exemplo Os sites que você acessa: Google, YouTube, Netflix, etc.



A Internet é a **infraestrutura**. A Web é um dos **serviços** que usamos todos os dias!

Breve História da Web

A Web **evoluiu** muito desde sua criação até as aplicações modernas que usamos hoje.



A Web continua evoluindo e você fará parte dessa história como **desenvolvedor(a) Front-End!**

Como Funciona um Site?

Quando você digita um endereço na barra do navegador, **muitas coisas acontecem** em frações de segundo.



Exemplo:
Você digita
www.google.com

O que acontece?

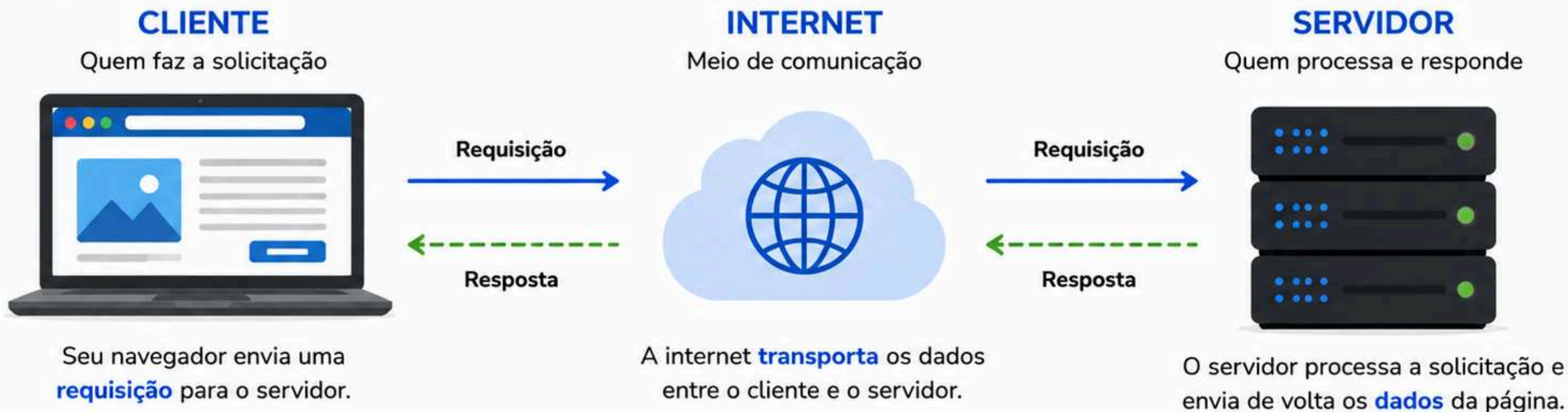
- 1 O navegador envia uma **requisição** para o servidor.
- 2 O servidor **processa** a solicitação.
- 3 O servidor envia os **dados** da página de volta.
- 4 O navegador **interpreta** o código (HTML, CSS, JS).
- 5 A página é **exibida** para você!



Tudo isso acontece em milissegundos e permite que você tenha **acesso a páginas, sistemas e aplicações incríveis** pela internet!

Arquitetura Cliente-Servidor

A Web funciona baseada no modelo **Cliente-Servidor**.
O navegador (**cliente**) faz solicitações e o **servidor** responde.



Em resumo:

O cliente **solicita**, o servidor **processa**, a internet **transporta** e o cliente **recebe**.

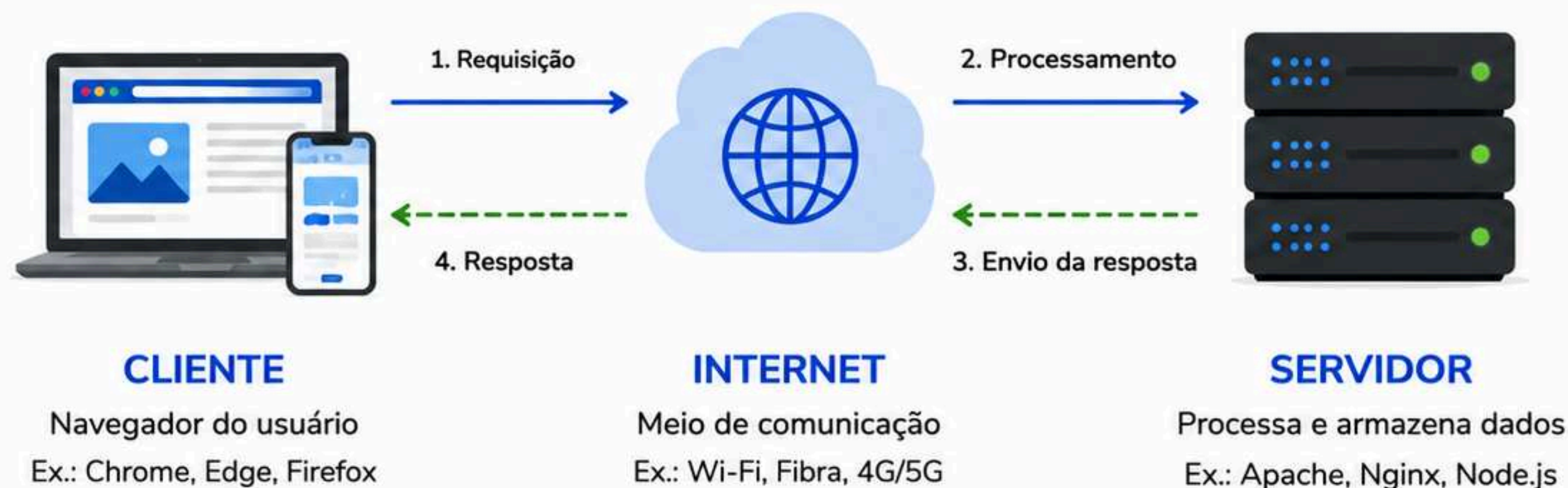
Arquitetura Cliente-Servidor

A arquitetura Cliente-Servidor é o modelo mais utilizado na Web.

O **cliente** faz solicitações e o **servidor** processa e retorna as informações.

Como acontece na prática?

- 1 O cliente faz uma **requisição**.
- 2 A requisição viaja pela internet até o **servidor**.
- 3 O servidor processa e envia a **resposta**.
- 4 A resposta volta e é **exibida** no navegador do cliente.



Exemplo:

Ao acessar www.unicesumar.edu.br, seu navegador (**cliente**) envia uma solicitação para o **servidor** da UniCesumar. O servidor processa seu pedido e retorna a página que você vê na tela.

Quem é o Cliente?

O **cliente** é quem faz as solicitações ao servidor.

Na Web, o cliente é o **navegador** e o dispositivo que você utiliza para acessar os sites e aplicações.

Exemplos de dispositivos (clientes)



Em resumo:

Você (usuário) utiliza um dispositivo com um navegador para acessar a Web.



Computador



Smartphone



Tablet



Desktop

Exemplos de navegadores (clientes)



Google Chrome



Mozilla Firefox



Microsoft Edge



Safari



Opera



Brave



Importante!

Cada navegador interpreta o código (HTML, CSS, JavaScript) e exibe a página para o usuário.

Esse é o papel do cliente na Web!

Quem é o Servidor?

O **servidor** é o responsável por **armazenar**, **processar** e **disponibilizar** as informações e serviços da Web.



Em resumo:

O servidor recebe as solicitações dos clientes, processa os dados e envia as respostas de volta pela internet.

O que um servidor faz?



Recebe as solicitações



Processa as solicitações



Acessa e gerencia os dados



Envia a resposta de volta ao cliente

Exemplos de servidores



Servidor Web
(Apache, Nginx, IIS)



Servidor de Banco de Dados (MySQL, PostgreSQL)



Servidor de E-mail
(Postfix, Exchange)



Servidor de Aplicação
(Tomcat, Node.js)



Importante!

Os servidores trabalham **24 horas por dia, 7 dias por semana**, para garantir que os sites e aplicações estejam sempre disponíveis para os usuários.

Como o Cliente e o Servidor se Comunicam?

A comunicação entre cliente e servidor na Web acontece por meio do protocolo **HTTP (HyperText Transfer Protocol)**.



HTTP é a linguagem da Web.

É ele que define como as mensagens são enviadas e recebidas entre o navegador (cliente) e o servidor.

Como funciona na prática?



Principais métodos HTTP

Método	Descrição
GET	Solicita dados de um recurso do servidor (ex.: carregar uma página).
POST	Envia dados para o servidor (ex.: formulário de cadastro).
PUT	Atualiza um recurso no servidor.
DELETE	Remove um recurso do servidor.

Exemplo de fluxo HTTP



Requisição (cliente → servidor)

```
GET /index.html HTTP/1.1
Host: www.unicesumar.edu.br
User-Agent: Mozilla/5.0 ...
```

O cliente envia a **requisição**



Resposta (servidor → cliente)

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
<html>...</html>
```

O servidor **responde** com os dados solicitados



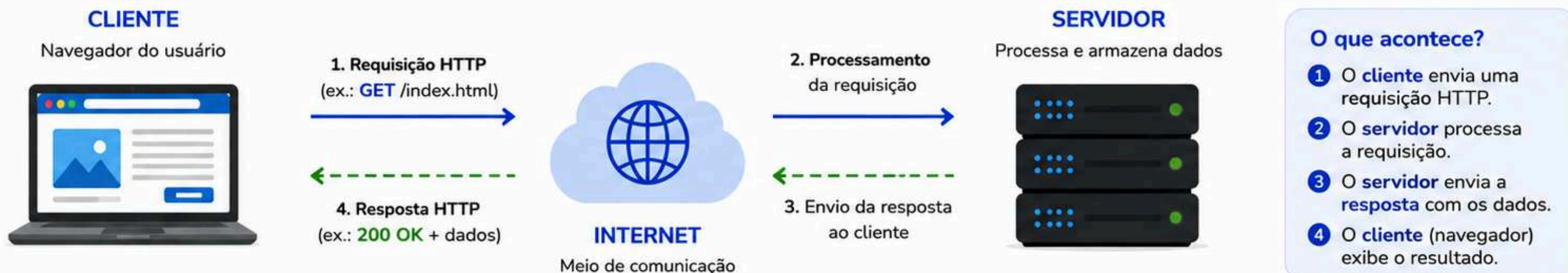
Importante!

O HTTP é um protocolo **sem estado** (*stateless*), ou seja, cada requisição é independente das anteriores.

Resumo: Comunicação Cliente-Servidor na Web

O **cliente** (navegador) solicita recursos ao **servidor** Web por meio do protocolo **HTTP**.

O servidor processa a solicitação e retorna a resposta.



Pontos-chave



- ✓ O **cliente** inicia a comunicação.
- ✓ O **servidor** responde às solicitações.
- ✓ A comunicação ocorre via **HTTP**.
- ✓ Esse modelo é a base de toda aplicação Web.

Exemplos do dia a dia



Ao acessar www.unicesumar.edu.br, o navegador (**cliente**) solicita a página ao servidor da UniCesumar.



Em uma loja online, o **cliente** envia dados de um pedido e o **servidor** processa e confirma a compra.



Ao enviar um e-mail, o **cliente** envia a mensagem e o **servidor** de e-mail a armazena e entrega ao destinatário.



Em resumo:

Cliente e **servidor** trabalham juntos para que a Web funcione. O cliente **solicita**, o servidor **processa** e **responde**!

Detalhando a Comunicação na Web

A comunicação Cliente-Servidor na Web acontece por meio de **requisições HTTP** e **respostas HTTP**, que seguem um formato padrão de mensagens.



O que acontece?

- 1 O **cliente** envia uma requisição HTTP para o servidor.
- 2 O **servidor** processa a requisição e acessa recursos (arquivos, banco de dados, etc.).
- 3 O **servidor** envia uma resposta HTTP de volta ao cliente.
- 4 O **cliente** recebe a resposta e o navegador exibe o resultado para o usuário.

Estrutura de uma Requisição HTTP

```

GET /index.html HTTP/1.1
Host: www.unicesumar.edu.br
User-Agent: Mozilla/5.0 ...
Accept: text/html
Accept-Language: pt-BR
Connection: keep-alive
  
```

(corpo da mensagem - opcional)

Linhas de identificação:
 - Linha de requisição: `GET /index.html HTTP/1.1`
 - Cabeçalhos (headers): `Host: www.unicesumar.edu.br`, `User-Agent: Mozilla/5.0 ...`, `Accept: text/html`, `Accept-Language: pt-BR`, `Connection: keep-alive`
 - Corpo (opcional): (corpo da mensagem - opcional)

Estrutura de uma Resposta HTTP

```

HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Content-Length: 1256
Date: Wed, 15 May 2024 14:30:00 GMT
Server: Apache/2.4.54 (Ubuntu)
Connection: keep-alive
<html>...</html>
  
```

Linhas de identificação:
 - Linha de status: `HTTP/1.1 200 OK`
 - Cabeçalhos (headers): `Content-Type: text/html; charset=UTF-8`, `Content-Length: 1256`, `Date: Wed, 15 May 2024 14:30:00 GMT`, `Server: Apache/2.4.54 (Ubuntu)`, `Connection: keep-alive`
 - Corpo (dados retornados): `<html>...</html>`



Importante!

O protocolo HTTP é baseado no modelo **requisição-resposta** e é **stateless** (sem estado): cada requisição é independente das anteriores.



Em resumo

O **cliente** (navegador) inicia a comunicação enviando uma **requisição HTTP**. O **servidor** processa, gera a **resposta** e envia os dados de volta, que o navegador exibe ao usuário.

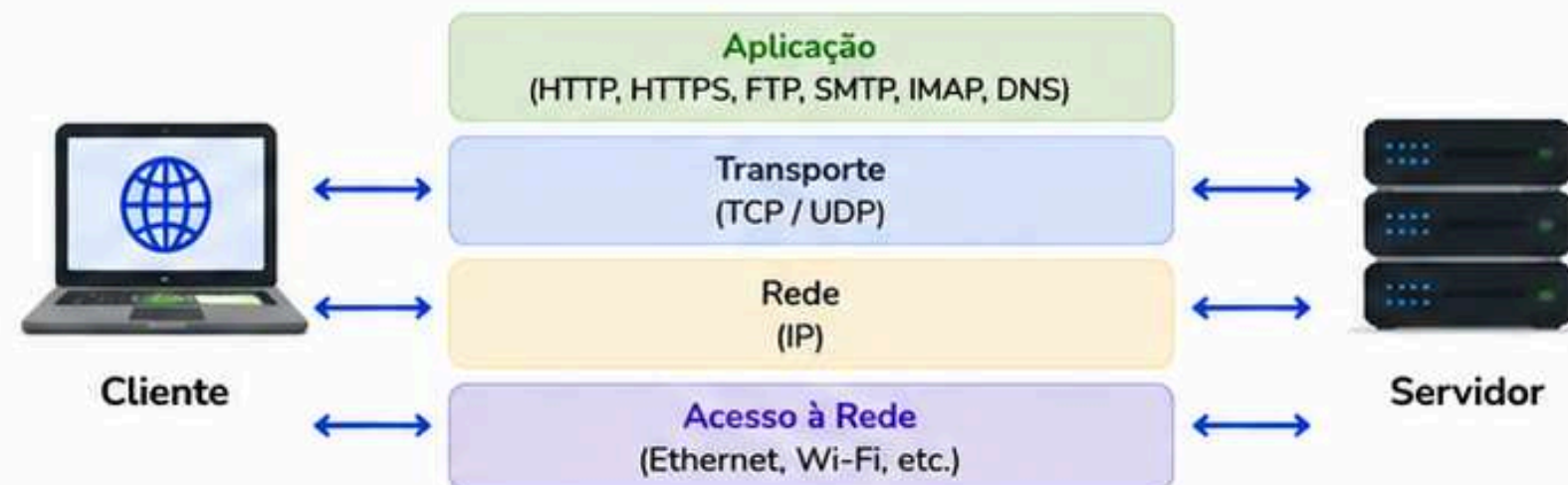
Protocolos e Padrões da Web

A Web funciona graças a um conjunto de **protocolos** e **padrões** que garantem a comunicação, a segurança e a interoperabilidade entre clientes e servidores.

Principais protocolos da Web

Protocolo	Descrição
 HTTP	Protocolo de aplicação usado para comunicação entre cliente e servidor. Opera na porta 80.
 HTTPS	Versão segura do HTTP que utiliza criptografia SSL/TLS. Opera na porta 443.
 FTP	Protocolo para transferência de arquivos entre cliente e servidor. Opera nas portas 20 e 21.
 SMTP	Protocolo usado para envio de e-mails entre servidores de correio. Opera na porta 25 (ou 587).
 IMAP	Protocolo usado para acesso e gerenciamento de e-mails no servidor. Opera na porta 143 (ou 993).
 DNS	Protocolo responsável pela resolução de nomes de domínio em endereços IP. Opera na porta 53.

Como os protocolos se relacionam?



Cada protocolo tem uma função específica e trabalha em conjunto para que a comunicação na Web seja **possível**.

Principais padrões da Web



W3C (World Wide Web Consortium): define os padrões para HTML, CSS, XML e outras tecnologias da Web.



HTML5: padrão para estruturação de páginas Web.



CSS3: padrão para estilização e layout das páginas Web.

Por que isso é importante?

- ✓ Garantem que diferentes sistemas possam se comunicar.
- ✓ Promovem segurança, confiabilidade e desempenho.
- ✓ Permitem que a Web evolua de forma organizada e padronizada.



Exemplo prático

Quando você acessa um site, seu navegador usa DNS para encontrar o servidor, HTTPS para se comunicar com segurança e HTTP para trocar as mensagens da aplicação.



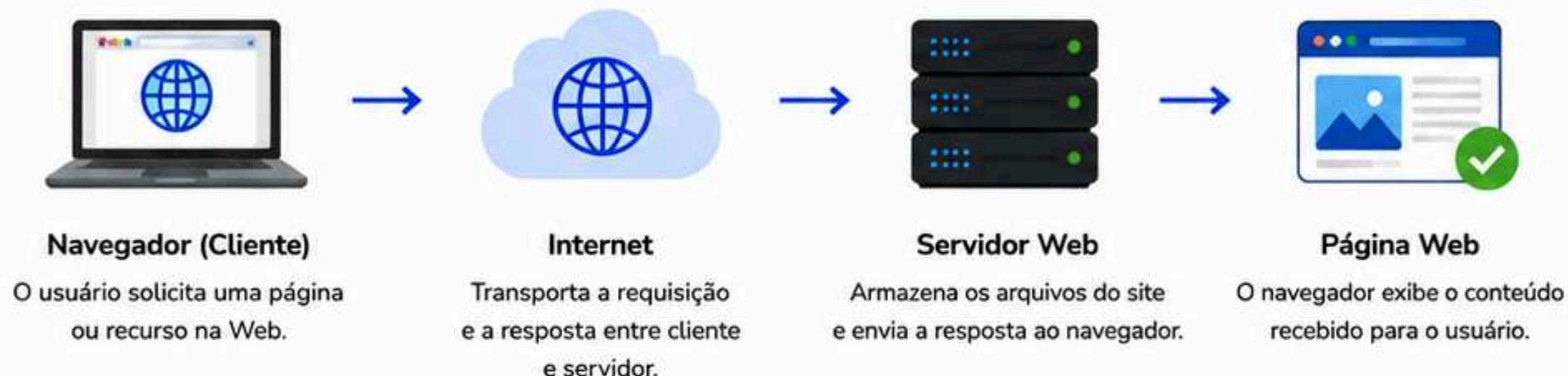
Em resumo:

Os **protocolos** definem as regras da comunicação, e os **padrões** garantem que todos falem a mesma língua na Web.

A Web (WWW) e Seus Elementos

A Web (**World Wide Web**) é um serviço que funciona sobre a Internet e permite o acesso a documentos e recursos interligados por meio de **links**.

Como a Web funciona?



Elementos principais da Web



URL: endereço único que identifica um recurso na Web.
Ex.: <https://www.unicesumar.edu.br>



HTML: linguagem de marcação usada para criar o conteúdo das páginas.



Hiperlinks: links que conectam documentos e recursos na Web.



Navegadores: software que permite acessar e visualizar páginas Web.

Estrutura de uma URL



Exemplo prático

← → ↻ 🔒 <https://www.unicesumar.edu.br/cursos/informatica>

Ao acessar essa URL:

- ✓ O navegador envia a requisição ao servidor da UniCesumar.
- ✓ O servidor localiza a página solicitada.
- ✓ O servidor envia a resposta (HTML, CSS, imagens, etc.).
- ✓ O navegador exibe a página para o usuário.



Por que a Web é importante?

- ✓ Permite o acesso rápido e fácil a informações do mundo todo.
- ✓ Integra diferentes tipos de mídia: texto, imagens, vídeos, áudio e muito mais.
- ✓ É a base de milhares de serviços e aplicações que usamos diariamente.



Em resumo:

A **Web** (WWW) utiliza a **Internet** para conectar clientes a **servidores** e disponibilizar informações em páginas interligadas por **links**.

HTML – O Esqueleto da Página

Todo documento HTML possui uma **estrutura básica** que define o início, o conteúdo e o fim da página.

O que é HTML?

HTML (*HyperText Markup Language*) é a linguagem de marcação usada para criar e estruturar o conteúdo das páginas Web.

Para que serve?

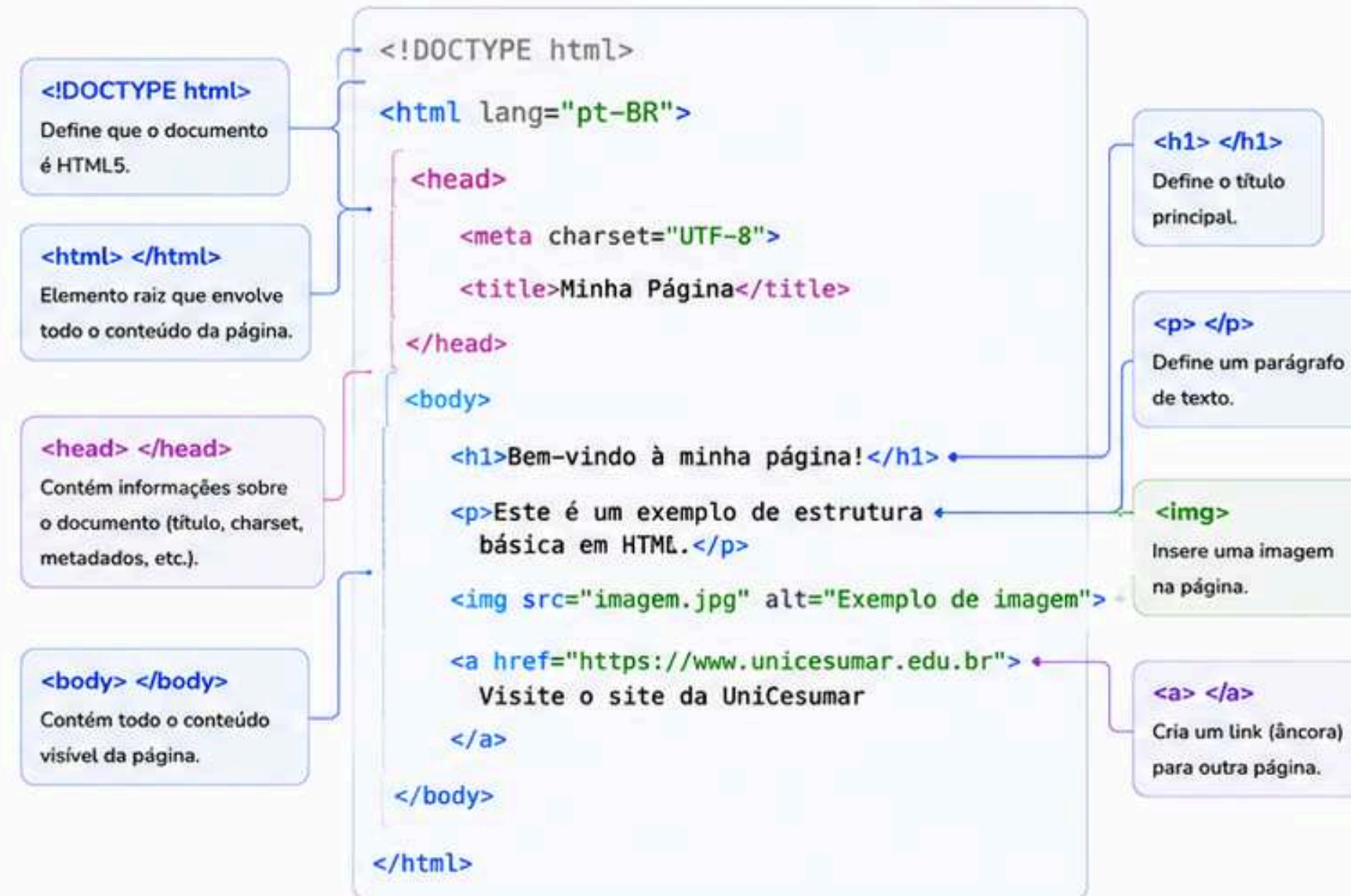
- ✓ Organiza o conteúdo da página.
- ✓ Define títulos, textos, imagens, links e outros elementos.
- ✓ É interpretado pelos navegadores.



Importante!

HTML define a **estrutura** do conteúdo, não a aparência. Para isso usamos CSS.

Estrutura básica de um documento HTML



Exemplos de elementos HTML



Títulos

```

<h1>Título principal</h1>
<h2>Subtítulo</h2>
<h3>Seção</h3>
  
```



Textos

```

<p>Este é um parágrafo de texto na página.</p>
  
```



Imagens

```


  
```



Links

```

<a href="https://www.unicesumar.edu.br">
  Acesse o site</a>
  
```



Em resumo:

HTML é o **esqueleto** da página Web. Ele organiza o conteúdo usando **elementos** como títulos, textos, imagens e links, que são interpretados pelos navegadores para exibição.

CSS – A Aparência

O CSS (Cascading Style Sheets) é a linguagem usada para definir o **estilo** e a **aparência** das páginas HTML.

O que é CSS?

CSS controla como os elementos HTML são exibidos na tela. Ele separa o **conteúdo** (HTML) da **apresentação** (aparência), tornando as páginas mais bonitas, organizadas e consistentes.

Para que serve?

- ✓ Define cores, fontes e tamanhos.
- ✓ Controla espaçamentos e alinhamentos.
- ✓ Cria layouts responsivos e modernos.
- ✓ Padroniza o visual de todo o site.



Importante!

HTML é o **esqueleto** (estrutura), CSS é a **roupa** (aparência). Juntos, eles formam uma página Web completa!

Exemplos do que o CSS pode controlar



Cores

Define as cores de texto, fundo, bordas e links.

```
body { background-color: #f0f7ff; }
h1 { color: #1e3a8a; }
a { color: #0a66c2; }
```



Fontes

Escolhe o tipo, tamanho, peso e estilo das fontes.

```
body { font-family: 'Roboto', sans-serif; }
h1 { font-size: 28px; font-weight: bold; }
p { font-size: 16px; line-height: 1.6; }
```



Espaçamentos

Controla margens, espaçamentos (padding) e alinhamentos.

```
div { margin: 20px; padding: 16px; }
h1 { margin-bottom: 16px; }
p { text-align: justify; }
```



Layout

Define o posicionamento e o layout dos elementos na página.

```
header { display: flex; justify-content: space-between; }
.container { width: 1000px; margin: 0 auto; }
.card { width: 30%; float: left; }
```

Antes e depois com CSS

Antes (apenas HTML)

Bem-vindo à minha página!

Este é um exemplo de página criada apenas com HTML.

[Visite o site da UniCesumar](#)



Informações importantes

- Aprender é transformar.
- Educação abre caminhos.
- UniCesumar com você.

Depois (HTML + CSS)

Minha Página Início Sobre Contato

Bem-vindo à minha página!

Este é um exemplo de página com HTML e CSS.

[Visite o site da UniCesumar](#) →



Informações importantes

- ✓ Aprender é transformar.
- ✓ Educação abre caminhos.
- ✓ UniCesumar com você.



Em resumo:

O **CSS** é responsável pela **aparência** da página Web, definindo **cores, fontes, espaçamentos** e **layout**. Ele torna o site mais atraente, organizado e fácil de usar.

JavaScript – A Inteligência

O JavaScript é a linguagem de programação que adiciona **interatividade** e **comportamento dinâmico** às páginas Web.

Exemplos do que o JavaScript pode fazer

O que é JavaScript?

JavaScript permite que as páginas Web respondam às ações do usuário em tempo real, sem precisar recarregar a página. Ele manipula o conteúdo HTML e o estilo CSS dinamicamente.

Para que serve?

- ✓ Torna a página interativa.
- ✓ Valida dados em formulários.
- ✓ Atualiza partes da página sem recarregar.
- ✓ Cria animações e efeitos visuais.
- ✓ Melhora a experiência do usuário.



Importante!

HTML é a **estrutura**, CSS é a **aparência** e JavaScript é a **inteligência** que faz acontecer!



Botões (eventos)

Executa ações quando o usuário clica em um botão.

```
document.getElementById('btn').addEventListener('click', function () {
  alert('Botão clicado!');
});
```



Formulários (validação)

Valida os dados informados pelo usuário.

```
const form = document.getElementById('form');
form.addEventListener('submit', function (e) {
  if (form.email.value === '') {
    e.preventDefault();
    alert('Informe seu e-mail!');
  }
});
```



Animações (efeitos)

Cria animações e transições para tornar a página mais atrativa.

```
const caixa = document.getElementById('caixa');
caixa.classList.add('animar');
// A classe .animar define a animação no CSS
```



Atualizações dinâmicas

Altera o conteúdo da página sem recarregar.

```
fetch('dados.json')
  .then(res => res.json())
  .then(dados => {
    document.getElementById('resultado').innerHTML = dados.mensagem;
  });
```

Exemplo de interação: antes e depois com JavaScript

Antes (sem JavaScript)

Lista de Tarefas

- Estudar HTML
- Aprender CSS
- Praticar JavaScript



Ao clicar em "Adicionar", nada acontece. A página não responde.

Depois (com JavaScript)

Lista de Tarefas

☒ Estudar HTML

☐ Aprender CSS

☐ Praticar JavaScript

☐ Criar um projeto


Ao clicar em "Adicionar", a tarefa é incluída na lista sem recarregar a página.



Em resumo:

JavaScript é a **inteligência** da página Web. Ele permite **interações**, **validações**, **animações** e **atualizações dinâmicas**, tornando o site mais funcional, moderno e agradável para o usuário.

Mercado de Trabalho

A área de desenvolvimento Web oferece diversas oportunidades de carreira em constante crescimento. Conheça alguns dos principais cargos:



Área em expansão

A transformação digital e a presença online das empresas aumentam a demanda por profissionais qualificados em desenvolvimento Web.

Principais cargos e suas responsabilidades



Desenvolvedor Front-End

Responsável por desenvolver a parte visual e interativa dos sites e aplicações.

- Cria interfaces responsivas e acessíveis
- Implementa layouts e estilos
- Garante a melhor experiência para o usuário

Tecnologias comuns:



Desenvolvedor Full Stack

Atua tanto no front-end quanto no back-end, desenvolvendo aplicações completas.

- Desenvolve interfaces e regras de negócio
- Trabalha com banco de dados e servidores
- Tem visão completa do projeto

Tecnologias comuns:



UX/UI Designer

Foca na experiência e na interface do usuário, garantindo que o produto seja útil, usável e agradável.

- Pesquisa e entende o usuário
- Cria protótipos e wireframes
- Define fluxos e interações
- Trabalha em conjunto com designers e desenvolvedores

Terramentas comuns:



Web Designer

Responsável pelo design visual dos sites, criando layouts atraentes e alinhados à marca.

- Cria o conceito visual
- Escolhe cores, tipografia e imagens
- Desenvolve layouts
- Colabora com desenvolvedores

Ferramentas comuns:



Desenvolvedor Mobile

Cria aplicativos para dispositivos móveis, como smartphones e tablets.

- Desenvolve apps nativos ou híbridos
- Integra dispositivos e APIs
- Publica e mantém aplicações nas lojas

Tecnologias comuns:



Habilidades valorizadas pelo mercado



Conhecimento técnico em tecnologias modernas



Trabalho em equipe e comunicação



Resolução de problemas e pensamento lógico



Aprendizado contínuo e adaptação



Foco no usuário e qualidade



Muito mais que código!

Profissionais da Web unem criatividade, tecnologia e empatia para criar soluções que transformam ideias em experiências digitais incríveis.



PRÓXIMA AULA

Aula 02

Criando Páginas com HTML



Estrutura HTML5



Tags



Formulários



Semântica



Na próxima aula você vai aprender:

Como criar a estrutura de uma página com HTML5, utilizar as principais tags, criar formulários e aplicar boas práticas de semântica.

Bons estudos!

Siga em frente com a



UniCesumar

EAD, SEMIPRESENCIAL E PRESENCIAL

www.unicesumar.edu.br